

TOP TIER ID — Project Summary & Developer Handoff

Document version: May 2026 **Prepared for:** Bryan Dell / TOP TIER ID **Live prototype:** <https://toptierid-web.vercel.app> **Repository:** <https://github.com/boosh711/toptierid>

1. Original Project Prompt

The following is the original request that initiated this project:

Build me a fully pieced together working prototype of the application described in the investor deck, and in the prototypes linked below. Let's start with it as a web based application for now but keep in mind that we will want to adapt it to a mobile app at some point in the near future as well. If there are any external connections needed, for example login services, just stub them out for now so we can add them in later.

>

The application is for helping high school athletes who want to get recruited to college sports have a single place for their profiles to be built and for coaches to have a single place to search for players with detailed filters, take notes, manage calendars during tournaments and scouting, and to contact players and parents through. It is effectively a two-sided marketplace between high school athletes and college coaches for recruiting, similar to how Handshake is a two-sided marketplace for college graduates to get connected to employers hiring for entry level jobs.

>

The application is essentially a shared database with different interfaces — one for the player, maybe a slightly different one for the player's parent in a supervisory capacity, and one for the coaches. It needs to be highly scalable to store video highlights from thousands of players, while also remaining performant so the database and architectural choices should keep that in mind.

>

Example Player flows: Sign up, Create profile, Messaging inbox, Analytics (see which coaches are looking at you), Add new highlights, Enter upcoming schedule (games, field, field number, jersey color).

>

Example Coach flows: Search and filter, Take notes on each player (typed or voice — private to coach or shared with org), Contact player function, Calendar with alerts for players you select to watch.

>

Design references: Investor deck in the "Top Tier ID" folder; Claude artifact prototypes (9 linked screens) for look, feel, and design.

2. Executive Summary

TOP TIER ID is a web-first recruiting marketplace prototype connecting high school girls' soccer athletes with college coaches. The product centers on a shareable **Digital ID** — a public athlete profile at `/p/[slug]` — backed by coach discovery, notes, messaging, scheduling, and analytics.

The implementation is a **Turborepo monorepo** deployed on **Vercel** with **PostgreSQL (Neon)** and **Prisma ORM**. Authentication, payments, email/SMS, and third-party OAuth are **stubbed** for rapid prototyping. The architecture is structured so a future **Expo/React Native** mobile app can share packages/types and packages/validators and consume the same API routes.

3. Repository & Deployment

3.1 GitHub

Item	Value
Repository	<code>https://github.com/boosh711/toptierid</code>
Default branch	<code>main</code>
Monorepo root	<code>top-tier-id/</code> (inside workspace <code>Top Tier ID/</code>)
Vercel root directory	<code>apps/web</code>

3.2 Vercel

Item	Value
Production URL	https://toptierid-web.vercel.app
Framework	Next.js 15 (App Router)
Build command	<code>cd ../../ && pnpm db:generate && pnpm db:push && pnpm --filter @top-tier-id/web build</code>
Install command	<code>cd ../../ && pnpm install</code>

Important: The Vercel build runs `pnpm db:push` automatically so Prisma schema changes (new columns) sync to Neon on every deploy. Without this, login and profile queries fail after schema updates.

3.3 Health Check

- **Endpoint:** `GET /api/health`
 - **Example:** <https://toptierid-web.vercel.app/api/health>
 - Returns `{ ok, database, userCount, envSource }` — use to verify DB connectivity and seed status.
-

4. Technology Stack

Layer	Technology
Frontend	Next.js 15, React 19, App Router, Tailwind CSS
Fonts	Inter (body), Russo One (display headings)
Brand color	#1E6BD6 (TOP TIER ID blue)
UI theme	Dark mode (black/navy surfaces)
Backend	Next.js Server Actions + API Routes
Database	PostgreSQL via Prisma 6
Hosting	Vercel
DB hosting	Neon (pooled connection string)
File storage	Vercel Blob (optional) + local public/uploads + inline data URLs fallback
Monorepo	Turborepo + pnpm workspaces
Auth	Cookie-based JWT sessions (jose) + bcrypt passwords (stub, not OAuth)
Validation	Zod (packages/validators)
Shared types	packages/types

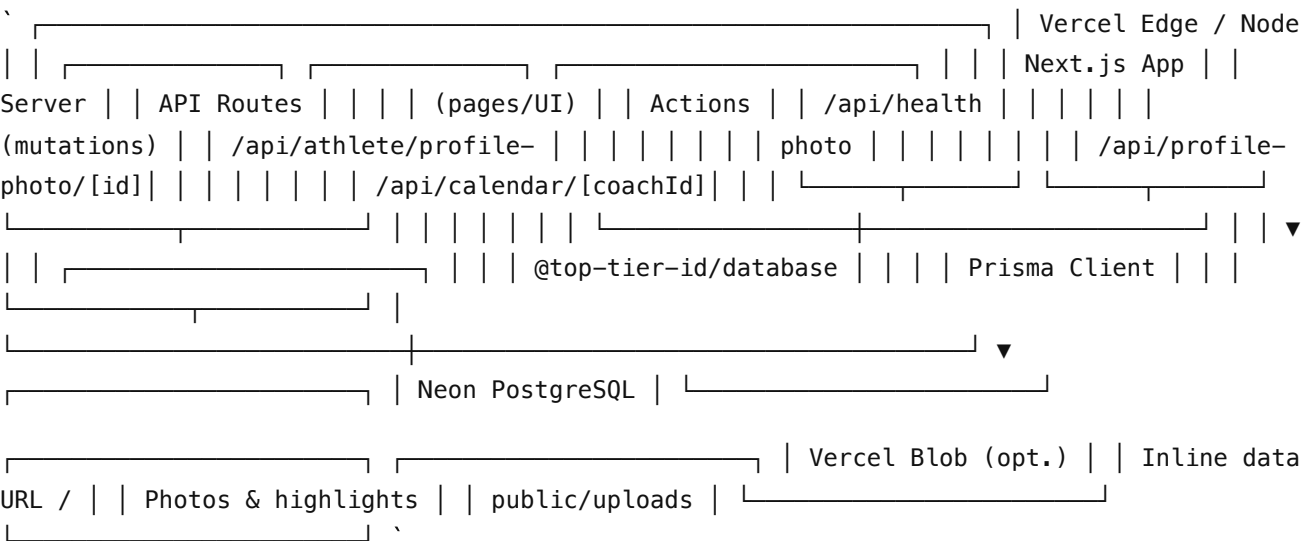
4.1 Prisma on Vercel

The monorepo uses `@prisma/nextjs-monorepo-workaround-plugin` and `binaryTargets` for `rhel-openssl-3.0.x` and `debian-openssl-3.0.x` so the query engine bundles correctly on Vercel serverless.

5. Monorepo Structure

```
` top-tier-id/ |— apps/ |  └─ web/ # Next.js 15 application (Vercel deploy target) | |— public/ | |  └─ logo.png # Official TOP TIER ID brand logo | |— src/ | |  |— app/ # App Router pages, layouts, API routes, actions | |  |— components/ # React UI components | |  └─ lib/ # Auth, storage, search, profile helpers | |— vercel.json # Vercel build/install overrides | |— next.config.ts |— packages/ | |— database/ # Prisma schema, migrations, seed | |— types/ # Shared constants (positions, divisions, brand) | |— validators/ # Zod schemas for server actions |— scripts/ | |— seed-production.sh # Pull Vercel env + db:push + db:seed | |— deploy-to-vercel.md # Deployment runbook |— docker-compose.yml # Local Postgres on port 5433 |— turbo.json |— pnpm-workspace.yaml |— README.md `
```

6. Architecture Overview



6.1 Request Flow (typical)

1. User hits a protected route (/athlete, /coach, /parent). 2. **Middleware** (apps/web/src/middleware.ts) verifies JWT cookie tti_session. 3. Server Component loads data via Prisma. 4. Mutations use **Server Actions** in apps/web/src/app/actions.ts or **API routes** for file uploads. 5. revalidatePath() refreshes cached pages after writes.

6.2 Role-Based Interfaces

Role	Base path	Primary capabilities
Athlete	/athlete	Onboarding, profile editor, highlights, schedule, inbox, analytics
Coach	/coach	Discover/search, My List, Staff Board, notes, calendar, messaging
Parent	/parent	Supervisory dashboard, linked athlete oversight, inbox, billing stub
Public	/p/[slug]	Shareable Digital ID (no auth required)

7. Database Schema (Prisma)

Location: packages/database/prisma/schema.prisma

7.1 Core Models

Model	Purpose
User	Email/password account with role (ATHLETE, PARENT, COACH)
AthleteProfile	Slug, stats, bio, photo, colors, social links, photo position, premium flag
CoachProfile	College, title, program affiliation
ParentLink	Links parent user to athlete profile
CollegeGoals	Divisions, regions, target schools
Highlight	Video URL, title, sort order
ScheduleEvent	Games with field, jersey color, tournament info
CoachFavorite	Coach "My List" entries
CoachNote	Typed/voice notes (PRIVATE or PROGRAM visibility)
ProfileView	Analytics — who viewed an athlete profile
MessageThread / Message	Inbox messaging
CalendarWatch	Coach watches athlete schedule events
ContactRequest	Coach → athlete contact flow
Program / ProgramMembership	Coach org / staff board

7.2 Athlete Profile Fields (notable)

- slug – unique public URL (/p/jordan-smith)
 - photoUrl – blob URL, local path, or base64 data URL
 - photoPositionX, photoPositionY – hero image focal point (0–100%)
 - primaryColor, secondaryColor, accentColor – Digital ID theming
 - instagramUrl, tiktokUrl, youtubeUrl, hudlUrl, xUrl – social links
 - isPublished, isPremium, onboardingStep
-

8. Routes & Pages

8.1 Public

Route	Description
/	Marketing landing + pricing
/auth/login	Email/password login
/auth/signup	Registration (athlete/coach/parent)
/auth/demo	Quick-login demo accounts
/p/[slug]	Public Digital ID profile

8.2 Athlete

Route	Description
/athlete	Dashboard + profile completion checklist
/athlete/onboarding	Multi-step wizard
/athlete/profile	Full profile editor (photo, colors, social, goals)
/athlete/highlights	Video upload manager
/athlete/schedule	Game schedule CRUD
/athlete/inbox	Messaging
/athlete/analytics	Profile views (premium reveals coach names)

8.3 Coach

Route	Description
/coach	Discover athletes with filters
/coach/list	Saved favorites
/coach/staff	Program staff board notes
/coach/schedule	Calendar watchlist + ICS export
/coach/inbox	Messaging
/coach/athlete/[slug]	Athlete detail + notes/actions

8.4 Parent

Route	Description
/parent	Supervisory dashboard
/parent/profile	Linked athlete overview
/parent/inbox	Messaging
/parent/billing	Billing stub

8.5 API Routes

Route	Method	Purpose
/api/health	GET	DB connectivity + user count
/api/auth/demo	GET	Demo quick-login redirect
/api/athlete/profile-photo	POST	Upload profile photo (multipart)
/api/profile-photo/[id]	GET	Serve stored photo (data URL / blob / local)
/api/calendar/[coachId]	GET	ICS calendar export for coach watchlist

9. Server Actions (Key Mutations)

File: apps/web/src/app/actions.ts

Action	Purpose
loginAction / signupAction	Auth
updateAthleteBasics	Position, GPA, bio, etc.
updateProfileStyle	Digital ID colors
updatePhotoPosition	Hero photo focal point
updateSocialLinks	Instagram, TikTok, YouTube, Hudl, X
updateCollegeGoals	Divisions, regions, target schools
uploadPhoto	Profile photo (legacy; editor uses API route)
uploadHighlight / deleteHighlight	Video reels
createScheduleEvent / deleteScheduleEvent	Schedule
toggleFavorite	Coach My List
saveCoachNote / saveVoiceNote	Coach notes (voice = stub transcription)
addCalendarWatch	Coach calendar
contactAthlete	Contact request + messaging
sendMessage	Thread replies
recordProfileView	Analytics tracking

10. Authentication

Stub implementation – not production OAuth.

- **Session cookie:** tti_session (httpOnly JWT, 7-day expiry)
- **Signing:** SESSION_SECRET env var (jose HS256)
- **Passwords:** bcrypt hashed in User.passwordHash
- **Middleware:** Protects /athlete/*, /coach/*, /parent/*

Future: Replace with Clerk, Auth0, or similar; interfaces are isolated in apps/web/src/lib/auth.ts.

11. File Storage

File: apps/web/src/lib/storage.ts

Environment	Behavior	
Local dev	Writes to public/uploads/{photos	highlights}/
Vercel + Blob token	Vercel Blob (@vercel/blob)	
Vercel without Blob	Profile photos stored as base64 data URLs in PostgreSQL; served via /api/profile-photo/[id]	

Photo upload flow: 1. Client compresses/converts (HEIC → JPEG via heic2any) 2. POST /api/athlete/profile-photo 3. DB stores photoUrl; display uses getProfilePhotoUrl() helper

Highlights: Video files use same storage provider; free tier limited to 1 highlight.

12. Environment Variables

Variable	Required	Description
DATABASE_URL	Yes	Neon pooled PostgreSQL URL
SESSION_SECRET	Yes	32+ char random string for JWT
NEXT_PUBLIC_APP_URL	Yes	e.g. https://toptierid-web.vercel.app
BLOB_READ_WRITE_TOKEN	Recommended	Vercel Blob for uploads
POSTGRES_PRISMA_URL	Alt	Auto-set by Vercel Postgres integration
POSTGRES_URL	Alt	Fallback DB URL detection

Local: Copy .env.example to apps/web/.env and packages/database/.env. Docker Postgres runs on port **5433**.

13. Demo Accounts & Seed Data

Password (all accounts): demo1234

Role	Email
Athlete	athlete@demo.com
Parent	parent@demo.com
Coach (Head)	coach.head@demo.com
Coach (Asst)	coach.asst@demo.com
Coach	coach2@demo.com, coach3@demo.com, coach4@demo.com

Public profile example: /p/jordan-smith **Quick login:** /auth/demo

13.1 Seeding

```
`bash
```

[Local \(Docker Postgres\)](#)

```
docker compose up -d pnpm db:push pnpm db:seed
```

Production (Neon)

```
export DATABASE_URL="neon-pooled-url" pnpm db:push pnpm db:seed
```

Or use helper script (requires Vercel CLI)

```
./scripts/seed-production.sh `
```

Seed file: packages/database/prisma/seed.ts – creates 35 users, demo athletes, coaches, messages, schedule events, etc.

14. Features Implemented

14.1 Athlete

- Multi-step onboarding wizard
- Profile editor: player info, photo upload (HEIC support), photo position drag/sliders, school color matching, accent/background swatches, college goals, social links

- Live preview of public Digital ID
- Public profile at `/p/[slug]` with tall 9:16 hero photo, gradient overlay, stats bar, about, highlights, schedule, college goals, connect icons
- Highlights upload (1 free / unlimited premium)
- Schedule with opponent, field, field #, jersey color, tournament
- Inbox messaging
- Analytics with premium gating (coach names hidden on free tier)

14.2 Coach

- Discover with search filters (position, grad year, GPA, state, club)
- My List (favorites)
- Staff Board (program-scoped notes)
- Per-athlete notes (typed + voice stub)
- Calendar watchlist with ICS export (`/api/calendar/[coachId]`)
- Contact athlete / messaging
- Profile view tracking

14.3 Parent

- Link to athlete by slug
- Supervisory dashboard
- Inbox
- Billing page (stub)

14.4 UI / Brand

- Dark mode theme matching Digital ID prototypes
 - Brand blue `#1E6BD6`
 - Official logo image (`public/logo.png`)
 - Russo One + Inter typography
-

15. Stubbed Integrations (Not Production-Ready)

Integration	Status	Notes
OAuth (Google/Apple)	Stub	Email/password only
Stripe payments	Stub	Parent billing page is UI only
Email / SMS	Stub	No transactional email
Whisper / voice transcription	Stub	Voice notes save audio URL only
Google/Apple Calendar OAuth	Stub	ICS export works
NCAA compliance enforcement	Partial	canCoachContactAthlete() grad-year gate

16. Local Development

Prerequisites

- Node.js 20+
- pnpm 9+
- Docker (for local Postgres)

Commands

```
`bash cd top-tier-id pnpm install cp .env.example apps/web/.env cp .env.example packages/database/.env docker compose up -d pnpm db:push pnpm db:seed pnpm dev `
```

Open <http://localhost:3000>

Useful scripts

Command	Purpose
pnpm dev	Start Next.js dev server
pnpm build	Production build (all packages)
pnpm db:generate	Regenerate Prisma client
pnpm db:push	Sync schema to database
pnpm db:seed	Seed demo data
pnpm db:studio	Prisma Studio GUI

17. Key Files for New Developers

Area	Path
Prisma schema	packages/database/prisma/schema.prisma
Seed data	packages/database/prisma/seed.ts
Server actions	apps/web/src/app/actions.ts
Auth	apps/web/src/lib/auth.ts
Middleware	apps/web/src/middleware.ts
Public profile UI	apps/web/src/components/public-profile.tsx
Profile editor	apps/web/src/app/athlete/profile/editor.tsx
Photo upload API	apps/web/src/app/api/athlete/profile-photo/route.ts
Storage abstraction	apps/web/src/lib/storage.ts
Zod validators	packages/validators/src/index.ts
Shared types/constants	packages/types/src/index.ts
Vercel config	apps/web/vercel.json
Tailwind theme	apps/web/tailwind.config.ts, apps/web/src/app/globals.css

18. Post-Build Enhancements (Session History)

After the initial prototype, the following were added iteratively:

1. **Dark mode UI** – Black/navy theme matching Claude artifact prototypes
2. **Brand blue theme** – #1E6BD6 accents across app
3. **Profile color picker** – School lookup + accent/background swatches for public Digital ID
4. **College goals editor** – Divisions, regions, target schools
5. **Profile photo upload** – With live preview, Vercel-compatible storage, HEIC support
6. **Official logo** – Image asset replacing text logo
7. **Social media links** – Instagram, TikTok, YouTube, Hudl, X with connect icons on public profile
8. **Hero photo redesign** – Tall 9:16 aspect ratio with bottom gradient
9. **Photo position controls** – Drag/slider to set hero focal point
10. **Auto db:push on Vercel build** – Prevents login failures after schema changes

19. Known Issues & Operational Notes

1. **Local .env points to Docker** (localhost:5433) – does not affect production; seed production separately. 2. **Vercel Blob optional** – Without BLOB_READ_WRITE_TOKEN, photos store as data URLs in Postgres (works but increases DB size). 3. **Prisma on Vercel** – Requires monorepo workaround plugin; do not remove from next.config.ts. 4. **Demo password** – demo1234 is hardcoded in seed; rotate for any public demo. 5. **Mobile app** – Not started; shared packages are ready for Expo integration.

20. Future Roadmap (Suggested)

- Replace stub auth with OAuth (Clerk/Auth0)
 - Stripe subscriptions for athlete premium (\$14.99/mo referenced in UI)
 - Real email notifications for messages/contact requests
 - Whisper API for voice note transcription
 - Vercel Blob required for production video at scale
 - Expo mobile app consuming shared packages + API routes
 - Proper Prisma migrations (vs. db push) for production schema versioning
 - CDN for public profile assets
 - Admin dashboard
-

21. Investor & Design References

- **Investor deck:** TOP-TIER-ID-Investor-Deck.pdf (parent workspace folder)
 - **Claude artifact prototypes:** 9 public artifact URLs provided in original prompt (Digital ID, coach discover, onboarding, etc.)
-

22. Contact & Handoff Checklist

For a new developer taking over:

- ☐ Clone <https://github.com/boosh711/toptierid>
- ☐ Request Vercel project access (or env vars export)
- ☐ Request Neon database access (or DATABASE_URL)
- ☐ Run local setup (Section 16)
- ☐ Verify /api/health on production
- ☐ Log in via /auth/demo with athlete@demo.com / demo1234
- ☐ Review Prisma schema and seed data
- ☐ Read apps/web/src/app/actions.ts` for mutation patterns
- ☐ Note stubbed integrations (Section 15) before building production features

End of document — TOP TIER ID Project Summary, May 2026